LEVVA · HUB DE ACELERAÇÃO

# Harness de Domínio

## Definição de produto

Documento de trabalho · nome provisório

Marcelo Pinheiro, Head of AI Engineering

9 de outubro de 2026

Documento confidencial para a diretoria da Levva

> Nome provisório. O produto será operado por uma empresa separada, financiada pela Levva. O nome definitivo será escolhido depois.

---

## 1. Em uma frase

O Harness de Domínio é a plataforma que registra os padrões de construção de software de uma empresa e os entrega a qualquer agente de IA no momento em que ele toma uma decisão. Antes da produção, a plataforma também verifica se esses padrões foram seguidos.

---

## 2. O problema

A IA colocou a construção de aplicações nas mãos da área de negócio. Pessoas sem formação técnica já constroem ferramentas com Claude Code, Cursor, Copilot, Lovable, Replit e similares. O resultado são aplicações que ignoram os padrões de governança, segurança, código, design e marca da empresa.

Hoje, as empresas lidam com isso de duas formas, e as duas são ruins:

- **Travar:** áreas de arquitetura e TI revisam tudo manualmente e viram gargalo. O negócio desiste ou passa a construir por fora.
- **Liberar:** cada pessoa configura seu próprio agente do seu jeito, e os padrões ficam espalhados e inconsistentes. Parte deles nem está documentada.

O gargalo não é capacidade técnica. Times de tecnologia saberiam montar uma solução caseira, mas estão absorvidos pela operação e ninguém tem o mandato de organizar e centralizar os padrões para consumo por agentes de IA.

**Evidência de demanda:** o Vini, gestor de tecnologia da Ticket, procurou a Levva querendo exatamente isso. Ele já tinha conectado o Claude Code ao Figma sozinho, sem dificuldade, mas continuou procurando uma forma de direcionar os agentes do time para os padrões da Ticket, começando pelo design system. A conexão técnica é fácil. O que falta é centralização, padronização e validação.

---

## 3. O que é o produto

Um **harness de domínio**: uma camada de inteligência corporativa que fica entre os padrões da empresa e os agentes de IA que constroem aplicações. Ela tem quatro funções:

1. **Registrar** os padrões da empresa em módulos estruturados.
2. **Orientar** os agentes durante a construção, via MCP, entregando o padrão certo na hora da decisão.
3. **Validar** as aplicações antes da produção, com uma etapa no pipeline de CI.
4. **Gerar inteligência** sobre como os padrões são consultados, seguidos e desviados.

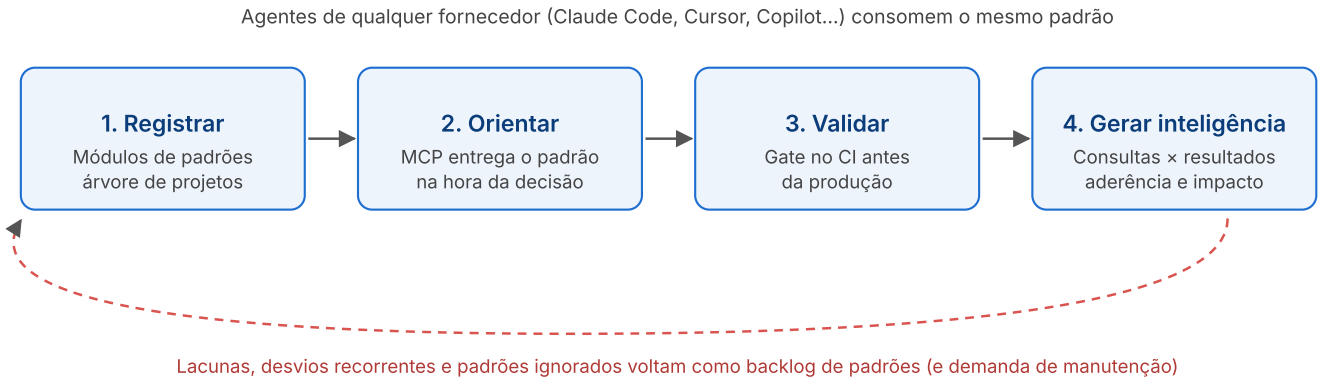



Figura 1 · As quatro funções do harness e o ciclo de aprendizado da governança

O harness atua sobre qualquer ferramenta. O mesmo padrão é servido para Claude Code, Cursor, Copilot ou qualquer agente compatível com MCP.

---

# 4. Para quem

| Papel | Quem é | O que quer |
|---|---|---|
| **Comprador** | Gestor de tecnologia, TI ou governança, em empresas com alguma maturidade, que já têm ou querem ter governança sobre aplicações do negócio | Impor padrões sem virar gargalo; visibilidade sobre o que está sendo construído |
| **Usuário principal** | Profissional de negócio que constrói aplicações com IA, geralmente sem familiaridade com MCP ou conceitos técnicos | Construir rápido sem ser barrado depois |
| **Donos de padrões** | Arquitetura, design, segurança, dados | Que seus padrões sejam seguidos sem revisar tudo manualmente |

**Duas formas de vender o mesmo produto:**

- **Controle (caso Ticket):** "saiba o que está sendo construído e garanta que segue os padrões".
- **Aceleração (hipótese Corteva):** áreas de arquitetura que hoje travam tudo por falta de confiança poderiam liberar mais se os padrões fossem verificados automaticamente. Nessa leitura, a arquitetura deixa de ser a área contornada e vira dona dos módulos e patrocinadora do produto. A métrica de venda passa a ser tempo de aprovação. Risco: a arquitetura enxergar a plataforma como ameaça ao seu papel.

Clientes com menor maturidade digital, sem pipeline nem governança, não são o foco inicial. O fluxo de nova aplicação (seção 5.3) pode atendê-los no futuro.

---

# 5. Como funciona

## 5.1 Módulos de padrões

A empresa registra seus padrões em módulos:

| Módulo | O que contém |
|---|---|
| **Design system** | Componentes, tokens, regras de uso de interface |
| **Stack de código** | Linguagens, frameworks, bibliotecas e versões aprovadas ou proibidas |
| **Domínios e entidades** | Mapa dos domínios de negócio e das entidades que existem na empresa |

| Módulo | O que contém |
|---|---|
| **Fontes de informação** | Onde estão os dados, quais fontes são autorizadas e para quê |
| **Marca** | Padrões de identidade visual e de comunicação |
| **Padrões gerais** | Demais convenções organizacionais (segurança, nomenclatura, processos etc.) |

Os padrões também ficam disponíveis via API, para que outros sistemas da empresa consumam a plataforma como fonte oficial.

### 5.2 Hierarquia de projetos

Os padrões são organizados numa **árvore de projetos**. Cada projeto pode ter um pai e filhos. A plataforma não impõe significado aos níveis: o cliente decide se um nó é uma unidade de negócio, uma tribo, um fluxo de valor ou uma aplicação.

- **Precedência:** em caso de conflito, vence o projeto mais específico, ou seja, o mais próximo da aplicação.
- **Regras travadas:** o projeto pai pode marcar regras como não sobrescrevíveis. Isso protege regras críticas, como as de segurança.
- **Controle de criação de filhos:** criar um projeto filho exige permissão. Assim, criar um filho não vira um atalho para fugir de regras travadas.

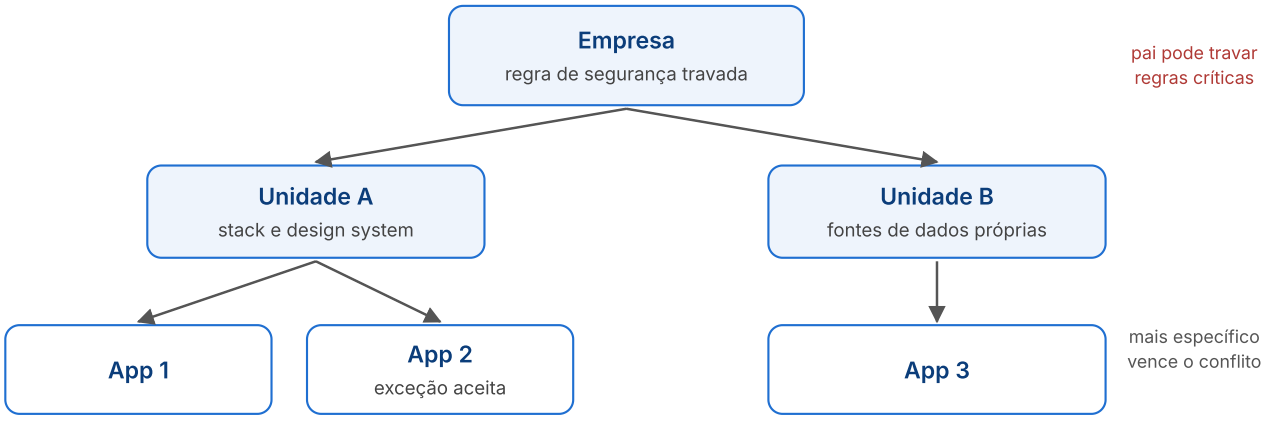



Figura 2 · Exemplo de árvore de projetos: precedência do mais específico e regras travadas pelo pai

### 5.3 Fluxo de nova aplicação

É a porta de entrada da pessoa de negócio:

1. A pessoa descreve o que quer construir na plataforma.
2. A aplicação é posicionada na árvore de projetos.
3. A plataforma gera um **boilerplate** já com a stack e o design system corretos, o **arquivo de identificação do projeto**, o MCP configurado e as skills instaladas.
4. A pessoa continua a construção na ferramenta que preferir, entre as que suportam esse fluxo.

Efeitos: a aplicação **nasce aderente**, em vez de depender só do agente consultar o padrão. E ela passa a existir para a TI desde o primeiro momento, não apenas quando chega ao gate.

### 5.4 Identificação do projeto

Cada aplicação tem um **arquivo oculto na raiz do repositório** (por exemplo, `.nome-do-projeto`) que declara a qual projeto da árvore ela pertence. O mesmo arquivo é usado:

- pelo MCP, para responder com as regras do ponto certo da hierarquia;
- pelo gate, para validar contra as regras certas;
- pela camada de inteligência, para ligar consultas e validações à mesma aplicação, o que é essencial para os relatórios cruzados e para a análise de impacto.

A integração com ferramentas sem repositório tradicional, como o Lovable, ainda precisa ser definida.

### 5.5 Orientação durante a construção (MCP)

O agente de construção se conecta ao MCP do harness e consulta os padrões por meio de **ferramentas granulares e bem descritas**, complementadas por **skills**.

- A plataforma **não empurra** padrões para dentro das ferramentas, por exemplo com hooks específicos de cada fornecedor, porque isso quebraria a neutralidade.
- Não há **briefing obrigatório** no início da sessão, porque consumiria tokens sem necessidade.
- Não há **revisão obrigatória** antes de o agente concluir, porque viraria um quality gate cansativo.
- Aceita-se que o resultado também depende da ferramenta, do agente e de quem opera. Por isso, **a qualidade das descrições das ferramentas é uma competência central do produto**.

**Dois níveis de resposta:**

| Nível | Resposta | Como funciona |
|---|---|---|
| **Gratuito / open source** | **Documento** | Devolve o conteúdo do padrão, e o agente interpreta |
| **Pago** | **Decisão** | O agente descreve a necessidade ("botão primário num formulário") e recebe só a resposta aplicável (componente X, token Y, exemplo de uso). Usa um agente de IA do lado do harness |

O agente de decisão é a inteligência que a empresa leva ao mercado, e a **acurácia** dele é um dos pilares do produto.

### 5.6 Validação antes da produção (gate no CI)

Uma etapa no pipeline de CI verifica se a aplicação segue os padrões do seu projeto.

- **Todo desvio vem com uma sugestão de correção** dentro do padrão, que pode ser entregue como instrução pronta para o usuário colar no próprio agente.
- **A plataforma não altera código.** Não há agente abrindo PR no repositório do cliente.
- **Quem decide é a TI.**
- **Desvios podem ser aceitos temporariamente**, mediante justificativa formal.
- Desvios aceitos geram **alertas recorrentes** a cada X dias, com o intervalo configurado pela TI.

### 5.7 Mudança de padrões

Quando um padrão muda:

- A plataforma mostra o **impacto**: quantas aplicações ficam desaderentes, quanto de aderência se perde e quais exceções são afetadas.
- As aplicações afetadas precisam de **aceite formal** para continuar passando no gate.
- Em vez de sugerir código, a plataforma pode criar **histórias ou tarefas** nas ferramentas de gestão que o cliente já usa.
- Ponto de atenção: prever **aceite em lote, por mudança**, para não exigir dezenas de aceites individuais a cada atualização de padrão.

## 6. Inteligência e relatórios

Com as duas pontas integradas, a plataforma enxerga a **intenção** (o que foi consultado durante a construção) e o **resultado** (o que chegou ao pipeline).

**Do lado da construção (MCP)**

- **Mapa de demanda:** quais domínios, entidades e fontes são mais consultados e onde o negócio está construindo.
- **Lacunas do harness:** perguntas sem padrão correspondente, que formam um backlog do que falta definir.
- **Padrões ignorados:** módulos nunca consultados. Geralmente indicam descrição ruim, não padrão ruim.
- **Inventário de construtores:** quais áreas, pessoas e ferramentas estão construindo.

**Do lado da validação (gate)**

- **% de aderência** por aplicação, área e módulo, e sua evolução.
- **Desvios mais frequentes**, por severidade.

**Cruzando os dois lados**

- **Prova de efeito:** aderência das aplicações construídas com o harness versus as construídas sem ele.
- **"Consultou, mas desviou":** padrão ambíguo, resposta ruim ou necessidade real que o padrão não cobre.
- **Desvio recorrente como sinal de padrão obsoleto:** a governança aprende com o uso.
- **Aplicações fantasmas:** aplicações que chegam ao gate sem nenhuma consulta registrada.
- **Análise de impacto** antes de publicar uma mudança de padrão.
- **Linhagem de dados:** quais aplicações tocam quais fontes, com valor para LGPD e segurança.

Vários desses relatórios (lacunas, padrões obsoletos, análises de impacto) geram demanda natural para os serviços de manutenção. **O software alimenta o pipeline de serviços.**

Ponto de atenção: registrar o que pessoas de negócio pedem aos agentes é sensível. É preciso definir cedo o que é registrado e quem tem acesso.

## 7. Arquitetura: modelo híbrido

| Onde | O que roda |
|---|---|
| **Ambiente do cliente** | MCP, gate, conteúdo dos padrões e qualquer dado sensível |
| **Ambiente da empresa** | Painel de gestão e camada de inteligência |
| **Opcional, no ambiente da empresa** | Agente de decisão (plano pago) |

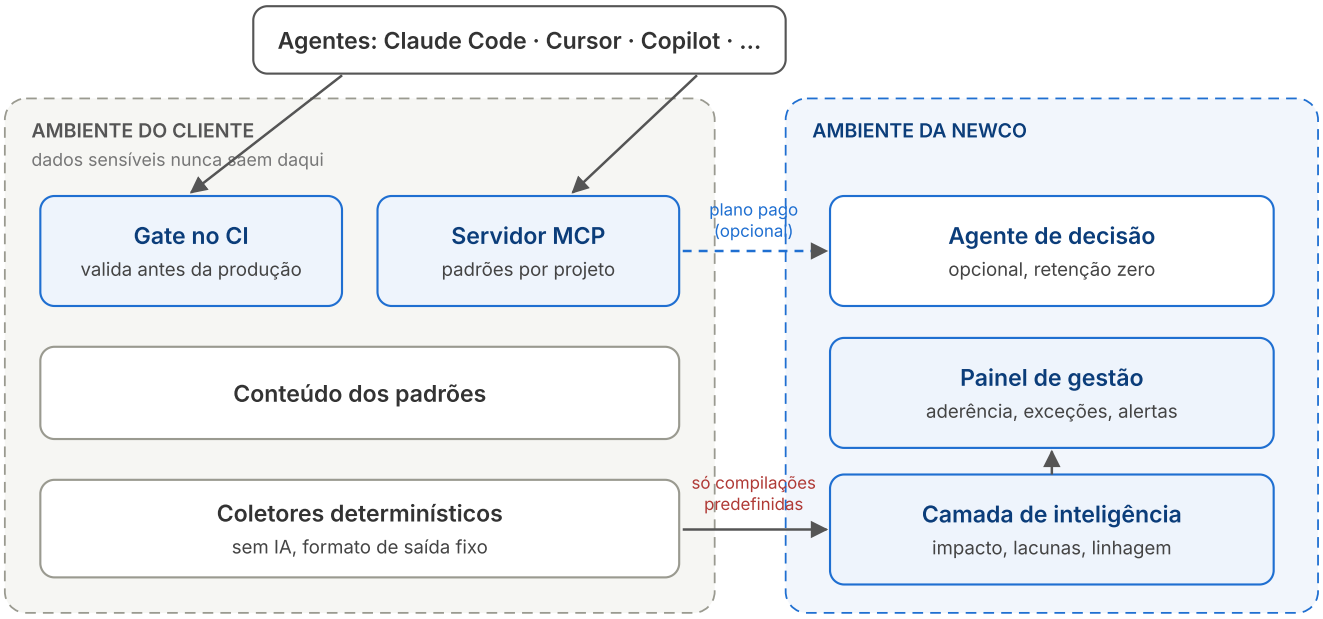



Figura 3 · Modelo híbrido: o que roda no cliente, o que roda na NewCo e o que atravessa a fronteira

- **Coletores no cliente:** componentes que podem ser determinísticos, sem IA. Eles percorrem padrões e resultados e enviam ao painel **apenas compilações predefinidas**. Como o formato do que sai é fixo e conhecido, a segurança do cliente consegue **auditar exatamente o que atravessa a fronteira**. Isso é também um argumento de venda.
- **Agente de decisão:** roda do lado da empresa, para os clientes que optarem por ativá-lo, com o argumento de economia de tokens e um **contrato de retenção zero**. A retenção zero precisa valer também para o provedor do modelo usado por baixo.

---

## 8. Por que uma plataforma, e não só um repositório de arquivos

Um time maduro poderia montar uma versão caseira com repositório Git, markdown, skills, regras do Cursor e um MCP open source. A plataforma se justifica por quatro motivos:

1. **Neutralidade:** um único padrão, servido para qualquer ferramenta. Configurar regras dentro de cada fornecedor (Cursor, Copilot, Claude) fragmenta os padrões e deixa de fora quem usa outra ferramenta.
2. **Responsabilidade:** existe uma empresa que responde pelo padrão e pela validação, e não uma pasta mantida por quem tiver tempo.
3. **Centralização com mandato:** o problema existe porque ninguém se dispõe a organizar. A plataforma e o serviço assumem esse papel.
4. **Inteligência:** o repositório guarda padrões. A plataforma mostra o que acontece com eles: consultas, aderência, desvios, impacto e lacunas.

---

## 9. Modelo de negócio

**Estrutura:** empresa separada, financiada pela Levva.

**Software**

- **Núcleo gratuito ou open source:** registro de padrões e MCP com resposta-documento. Funciona como funil.

- **Plano pago:** agente de decisão, inteligência e relatórios.

**Serviços**

- **Playbook** para quem quer levantar os próprios padrões, incluindo como mantê-los atualizados.
- **Implantação:** explorar as áreas e o negócio do cliente, encontrar os padrões existentes e, quando não existirem, propor como criá-los.
- **Manutenção contínua:** contratos por homem-hora ou homem-mês para atualizar artefatos.

**Como os padrões se mantêm atualizados**, numa combinação definida no playbook:

- a plataforma como fonte oficial dos padrões;
- integração com as fontes de origem (Figma, repositórios, catálogos de dados);
- um dono definido por módulo, com processo de revisão;
- o contrato de manutenção da empresa.

**O que a Levva ganha, mesmo que o produto não decole:** conhecimento prático e reaproveitável sobre como construir aplicações AI-core de produção, que geram valor com IA integrada, dentro dos padrões de um cliente.

---

## 10. Primeiro corte (MVP)

**Estratégia:** começar **estreito nos dois lados**. MCP e gate funcionando ponta a ponta, com poucos módulos. O gate prova o valor em números, e o MCP é o que tem apelo comercial na demonstração.

**Módulos do primeiro corte**

1. **Stack de código:** o mais fácil de validar automaticamente e fala direto com a dor da TI.
2. **Design system:** o melhor para demonstração e é exatamente onde começou o pedido real da Ticket.

**Próximo em prioridade:** fontes de informação, que é o módulo de maior valor de risco.

**Candidatos a piloto:** Ticket (demanda espontânea) e Corteva (hipótese de aceleração da área de arquitetura).

---

## 11. Princípios de produto

Decisões que se repetiram ao longo da conversa e que devem orientar as próximas:

- **A plataforma aponta e organiza; não altera o código do cliente.** Sugestões viram instruções para o agente do usuário ou tarefas na ferramenta de gestão do cliente, nunca PRs automáticos.
- **Neutralidade acima de conveniência.** Nada que dependa de um fornecedor específico.
- **Começar certo vale mais que corrigir depois.** O fluxo de nova aplicação e o boilerplate são o principal mecanismo de aderência.
- **A TI decide; a plataforma dá visibilidade.** Exceções, aceites e alertas são configurados pelo cliente.
- **O que é sensível fica no cliente.** Só compilações predefinidas e auditáveis saem do ambiente dele.

---

## 12. Riscos e pontos de atenção

- **Fornecedores das ferramentas** oferecendo regras e skills em nível de organização. A resposta é a neutralidade, e ela precisa ser defendida no produto.

- **Responsabilidade contratual:** "uma empresa que eu posso culpar" vira, no contrato, a questão de até onde vai a responsabilidade se uma aplicação aprovada pelo gate tiver uma falha de segurança.
- **Acurácia do agente de decisão:** se o harness recomendar algo que o gate reprova, a confiança do cliente se perde. É preciso medir e provar a qualidade das decisões.
- **Gate rígido demais:** se bloqueia demais, o negócio volta a publicar por fora. Se bloqueia de menos, vira relatório que ninguém lê.
- **Aplicações fora do pipeline:** aplicações publicadas direto em Lovable, Vercel ou Replit não passam pelo gate.
- **Harness desatualizado:** é pior que nenhum, porque o agente passa a impor com confiança um padrão abandonado.
- **Privacidade:** o registro do que usuários de negócio pedem aos agentes precisa de regras claras.
- **Escopo:** a visão completa é grande. Cada forma de manter os padrões atualizados e cada módulo é quase um produto à parte, por isso a disciplina no primeiro corte é essencial.

---

## 13. Decisões em aberto

- **Métrica principal de venda:** % de aplicações aderentes ou tempo de aprovação (caso Corteva).
- **Integração com Lovable** e outras ferramentas sem repositório tradicional.
- **Operação de qualidade do agente de decisão:** avaliações por cliente validadas pelos donos dos módulos, métrica de "decisão recomendada versus resultado no gate" e ciclo de feedback quando o usuário contesta uma resposta.
- **Limites de responsabilidade contratual.**
- **O que exatamente os coletores enviam** ao painel: lista das compilações predefinidas.
- **Política de registro e privacidade** das interações dos usuários com os agentes.
- **Nome** da empresa e do produto.

---

## 14. Próximo passo

Levar a proposta à diretoria da Levva. Para isso, falta definir:

- **O pedido:** investimento, pessoas e prazo para construir o primeiro corte e rodar um piloto.
- **O critério de sucesso do piloto:** por exemplo, um cliente pagante, um % de aderência medido num cliente real ou uma carta de intenção da Ticket.
- **O argumento de risco reduzido:** o que a Levva ganha mesmo que o produto não decole (seção 9).